Approfondissements

Scène strasbourgeoise, glitch art, feedback,
Pure Data, code libre

Support de cours — TOPLAP Strasbourg

Table des matières

1 Scène strasbourgeoise et glitch art en profondeur **3**
- 1.1 La scène strasbourgeoise . . . 3
- 1.2 Glitch art en profondeur . . . 4
  - 1.2.1 Origines . . . 4
  - 1.2.2 Théoriciens . . . 4
  - 1.2.3 Techniques — panorama . . . 5
  - 1.2.4 Glitch et live coding : ce qui s'aligne . . . 7
  - 1.2.5 Pour aller plus loin . . . 7

2 Le feedback en profondeur **7**
- 2.1 Origine cybernétique . . . 7
- 2.2 Feedback audio . . . 8
  - 2.2.1 Feedback acoustique : le Larsen . . . 8
  - 2.2.2 Alvin Lucier, *I Am Sitting in a Room* (1969) . . . 8
  - 2.2.3 Stockhausen, *Mikrophonie I* (1964) . . . 8
  - 2.2.4 No-input mixer (Toshimaru Nakamura) . . . 8
  - 2.2.5 Feedback synthétique : les bouclages internes . . . 8
- 2.3 Feedback vidéo . . . 9
  - 2.3.1 Le feedback analogique : caméra-écran . . . 9
  - 2.3.2 Feedback numérique : Hydra . . . 9
  - 2.3.3 Concept clé : le coefficient de fuite . . . 10
- 2.4 Feedback en code : auto-modification . . . 10
- 2.5 Feedback comme matière mathématique . . . 11
  - 2.5.1 La carte logistique (May, 1976) . . . 11
  - 2.5.2 L'attracteur de Lorenz (1963) . . . 11
- 2.6 Feedback : pratique pédagogique . . . 11
- 2.7 Pour aller plus loin . . . 12

3 Pure Data : programmation visuelle pour le temps réel **12**
- 3.1 Présentation . . . 12
- 3.2 Filiation : Patcher, Max, Pd . . . 12
- 3.3 Versions . . . 12
- 3.4 Concepts fondamentaux . . . 13
  - 3.4.1 Patch . . . 13
  - 3.4.2 Types de boîtes . . . 13
  - 3.4.3 Inlets et outlets . . . 13

3.4.4 Audio rate vs message rate . . . 13
3.5 Premier patch : un oscillateur . . . 13
3.6 Patch légèrement enrichi : un FM minimal . . . 14
3.7 Feedback en Pd . . . 14
3.8 GEM et le visuel . . . 14
3.9 Cas d'usage typiques . . . 14
3.10 Pure Data et live coding : le faux opposé . . . 15
3.11 Place dans le cursus . . . 15
3.12 Pour aller plus loin . . . 15

4 Open source, art libre et Creative Commons 16
4.1 Free software : l'origine politique . . . 16
4.1.1 Richard Stallman et le projet GNU (1983) . . . 16
4.1.2 Copyleft et licence GPL . . . 16
4.2 Open source : la reformulation pragmatique (1998) . . . 16
4.3 Familles de licences logicielles . . . 16
4.3.1 Copyleft fort . . . 16
4.3.2 Copyleft faible . . . 17
4.3.3 Permissif . . . 17
4.4 Les outils du cursus, leurs licences . . . 17
4.5 Creative Commons : libérer les œuvres, pas les programmes . . . 17
4.5.1 Critique des clauses NC et ND . . . 18
4.6 Free Art License (LAL) . . . 18
4.7 Open art, art libre, art en partage . . . 18
4.8 Limites et tensions . . . 18
4.9 Qu'enseigner concrètement . . . 19
4.10 Pour aller plus loin . . . 19

1 Scène strasbourgeoise et glitch art en profondeur

1.1 La scène strasbourgeoise

La scène live coding strasbourgeoise s'organise autour de **TOPLAP Strasbourg**, antenne locale de l'organisation internationale TOPLAP fondée à Hambourg en 2004. Le format de diffusion principal est l'évènement **Chaos Lab**, série de soirées qui réunissent live codeurs, vidéastes en temps réel et public dans une logique d'algorave francophone : code projeté, musique dansante générée à la volée, image en direct.

Chaos Lab

À ce jour, neuf éditions au moins ont eu lieu (la première très tôt dans l'histoire du collectif, la plus récente en novembre 2025). Chaque édition rassemble plusieurs performeurs sur une même soirée, dans une logique de *plateau partagé* plutôt que de tête d'affiche. Le public est invité à voir le code, à poser des questions, et la plupart des sets sont captés vidéo et archivés.

Ce qui caractérise Chaos Lab

- Un format hybride : performances individuelles, duos, et pièces collectives parfois improvisées entre live codeurs et VJ.
- Une attention pédagogique : pré-soirée parfois ouverte, ateliers en parallèle.
- Une diffusion en « cadavre exquis » parfois pratiquée où chaque performeur reprend la matière du précédent.

Performeurs récurrents La scène locale réunit notamment des projets comme *Crash Server*, *Beryann Parker*, *Randy Edgewood*, *PapiroveHouby*, *Pierre Danger*. Chacun a son territoire esthétique, du noise sombre à la techno algorithmique, du visuel abstrait au glitch revendiqué.

Lieux et écosystèmes proches

Strasbourg dispose d'un écosystème arts numériques actif :

- **Le Shadok**, fabrique du numérique de l'Eurométropole : ateliers, résidences, lieu d'accueil pour des évènements arts numériques.
- **HEAR** (Haute École des Arts du Rhin), notamment ses cursus arts plastiques et son atelier didactique visuel : terrain pour les expérimentations de jeunes artistes, croisements possibles avec le live coding.
- **Ososphère**, festival historique de musiques électroniques et arts numériques (depuis 1997) : scène de diffusion potentielle, public sensibilisé.
- **La Laiterie**, **Stimultania**, et autres lieux qui programment ponctuellement des artistes proches.

Sociologie de la scène

Trois traits pour comprendre cette scène :

1. **Une pratique transverse**. Les Chaos Labs croisent musiciens noise, codeurs exdéveloppeurs, plasticiens, danseurs ponctuellement. Ce n'est pas un milieu monolithique.

2. **Un fonctionnement DIY**. Pas de subvention massive, beaucoup d'auto-organisation, soirées montées avec peu de moyens dans des lieux variés.
3. **Une culture du partage**. Captations en ligne, code parfois publié, ateliers gratuits ou à prix libre. C'est cohérent avec la culture open source du live coding international (cf. partie 4 de ce document).

1.2 Glitch art en profondeur

Le glitch est l'esthétique du dysfonctionnement assumé. Plutôt que de masquer l'erreur, on l'invite, on la provoque, on l'organise. C'est l'une des esthétiques les plus directement compatibles avec le live coding parce qu'elle partage la même éthique : le code visible, le bug visible, la machine qui montre ses entrailles.

1.2.1 Origines

Nam June Paik, *Magnet TV* (1965). Un téléviseur cathodique avec un aimant posé dessus. L'aimant déforme le faisceau d'électrons, l'image télévisée se tord, perd sa cohérence. C'est, sans le mot encore, un acte glitch : prendre un signal industriel destiné à être « propre » et le perturber physiquement.

Steina et Woody Vasulka. À partir des années 1970, ils explorent systématiquement les artefacts vidéo : *drop-out*, *tearing*, déplacements de balayage, mises en boucle de signaux. Leur travail transforme ces accidents en grammaire.

Émergence du terme. Le mot *glitch* (« couac », « loupé ») s'impose dans la culture numérique des années 1990, d'abord côté musique (Oval, Pole, Alva Noto) où l'on intègre les CD rayés et les artefacts MP3 dans la composition, puis côté arts visuels.

1.2.2 Théoriciens

Rosa Menkman. *Glitch Studies Manifesto* (2010) puis *The Glitch Moment(um)* (2011). Elle pose la distinction fondatrice :

— **Glitch (pur)** : un accident réel, non reproductible, lié à un dysfonctionnement matériel ou logiciel.
— **Glitch-alike (esthétique du glitch)** : la reproduction délibérée et contrôlée de cette esthétique, par des artistes qui en maîtrisent les codes.

Cette distinction n'est pas un jugement de valeur (le glitch-alike n'est pas « moins bien ») mais une clarification critique : tous les glitchs ne se valent pas politiquement, parce que le *rapport au hasard et au contrôle* y est différent.

Iman Moradi. Sa thèse *Glitch Aesthetics* (2004) est l'un des premiers travaux universitaires sur le sujet. Distingue *pure glitch* et *glitch-alike* avant Menkman, et propose une typologie des artefacts.

Hugh S. Manon & Daniel Temkin. *Notes on Glitch* (*World Picture*, 2011). Texte court et puissant qui défend l'idée que le glitch est une expérience du *réel matériel* de l'image numérique : un rappel que derrière la fluidité de l'interface, il y a des bits, des protocoles, des conventions fragiles.

1.2.3 Techniques — panorama

Voici une typologie des techniques, classées par couche d'intervention.

1. Databending — corruption à l'octet près. On ouvre un fichier image (BMP, RAW, TIFF) avec un éditeur audio (Audacity), on lui applique des effets sonores (réverb, écho, delay, distortion), on le ré-enregistre. Le fichier reste valide « en théorie » (les en-têtes sont préservés si on ne touche qu'aux données pixel) mais visuellement il est tordu.

```
# Exemple : ouvrir un BMP dans Audacity
# 1. File > Import > Raw Data
#    Encoding: U-Law / 8-bit / mono
# 2. Sélectionner la zone après l'en-tête (54 octets pour BMP)
# 3. Appliquer un effet : Echo, Reverb, Phaser
# 4. Export > Other uncompressed > même format brut
# 5. Renommer en .bmp, ouvrir
```

2. Datamoshing — exploitation de la compression vidéo. Les codecs vidéo comme H.264 distinguent les images-clés (*I-frames*, image complète) et les images-différences (*P-frames*, qui n'encodent que la différence avec la précédente). Le datamosh consiste à *supprimer les I-frames* ou les remplacer par celles d'une autre vidéo : les P-frames suivantes « appliquent leur différence » à une image incohérente, créant des écoulements et des morphings caractéristiques.

```
# Avec ffmpeg, brutalement :
ffmpeg -i video1.mp4 -bsf:v "h264_metadata=display_orientation=0" \
       -c copy intermediate.h264

# Outils dédiés : avidemux (suppression manuelle de I-frames),
# AviGlitch (Ruby), datamoshing.com (web).
```

Référence artistique : Takeshi Murata *Monster Movie* (2005), Chairlift *Evident Utensil* (clip, 2008), Kanye West *Welcome to Heartbreak* (2009).

3. Pixel sorting. Algorithme popularisé par **Kim Asendorf** (2010). On parcourt l'image ligne par ligne (ou colonne par colonne) ; pour chaque segment vérifiant un critère (luminosité dans un intervalle, teinte, etc.), on trie les pixels selon une mesure (luminance par exemple). Résultat : des coulures verticales ou horizontales très reconnaissables.

```
# Pseudo-code (Python + PIL)
from PIL import Image
img = Image.open('input.jpg').convert('RGB')
w, h = img.size
pix = img.load()
for y in range(h):
    row = [pix[x, y] for x in range(w)]
    # Tri d'une portion de ligne par luminance
    start, end = 100, 500
    row[start:end] = sorted(
        row[start:end],
        key=lambda c: 0.299*c[0] + 0.587*c[1] + 0.114*c[2]
    )
    for x, p in enumerate(row):
```

```
        pix[x, y] = p
img.save('output.png')
```

Variantes : tri par bandes détectées par un seuil de luminance (look « Asendorf » canonique), tri en diagonale, tri par teinte.

4. Mauvaise interprétation de codec / format. On force un programme à lire un fichier comme s'il était dans un autre format. Exemple historique : ouvrir un `.exe` avec Photoshop en lui faisant croire qu'il s'agit d'un RAW ; ou interpréter un fichier audio comme image (ImageMagick : `convert -size WxH -depth 8 RGB:audio.wav out.png`).

5. Bit shifting et XOR. Manipuler les octets d'un fichier image en appliquant un décalage de bits ou un XOR avec une clé. Des formats comme PNG sont robustes (CRC) et refusent l'altération ; BMP, TIFF, RAW se laissent faire.

```
# Python : XOR sur les pixels d'un BMP (saute les 54 octets d'en-tête)
with open('in.bmp', 'rb') as f: data = bytearray(f.read())
key = 0x37
for i in range(54, len(data)):
    data[i] ^= key
with open('out.bmp', 'wb') as f: f.write(data)
```

6. JPEG — coups dans la quantification. Le format JPEG quantifie les coefficients DCT par bloc 8×8. En modifiant les tables de quantification (avec un éditeur hex ou un script), on obtient des artefacts de blocs caractéristiques. Le projet *jpglitch* (Daniel Temkin) automatise ce genre de manipulation.

7. Compression à outrance et générations. Sauver, recompresser, sauver, recompresser : la perte se cumule. *I Am Sitting in a Room* de Lucier appliqué à l'image (cf. section feedback). Des artistes (Jamie Beck, Constant Dullaart) ont fait des vidéos de ce processus.

8. Glitch GLSL en temps réel. Plutôt que de corrompre des fichiers, on écrit des shaders qui simulent le glitch. C'est ce que fait Hydra avec `pixelate`, des modulations brusques, du noise haute fréquence. Pour la performance, c'est l'approche de référence.

```
// Hydra : un glitch numérique
osc(40, 0.1)
  .modulateScale(noise(2, 1).pixelate(20, 20), 0.3)
  .pixelate(50, () => 50 + a.fft[0]*200)
  .modulate(noise(10).thresh(0.5))
  .out()
```

9. Glitch matériel : circuit bending. **Reed Ghazala**, années 1970–90 : court-circuiter volontairement des jouets sonores et des synthétiseurs bon marché pour en faire des instruments aux comportements imprévus. C'est le glitch *physique*, par opposition au glitch logiciel. Strasbourg a une communauté liée (à creuser localement).

10. Mojibake et corruptions de texte. La corruption typographique (encodages mélangés UTF-8/ISO-8859-1, etc.) produit ces séquences de caractères absurdes que tout

francophone qui a ouvert un email mal encodé connaît. Quelques artistes les utilisent comme matière (Jodi, Daniel Temkin avec *Internet Directory*).

1.2.4 Glitch et live coding : ce qui s'aligne

Le live codeur partage avec le glitch artist :

- **Le code visible.** Le glitch montre la matière numérique, le live coder montre l'instruction. Les deux refusent la magie noire de l'interface.
- **Le risque.** Une performance live coding peut crasher ; un glitch artist provoque le crash. Le crash est l'évènement, pas l'interruption.
- **La critique de la « propreté » numérique.** Refuser le rendu lisse, le 4K parfait, l'effet Hollywood, au profit d'une matière brute.

C'est pourquoi une initiation au live coding visuel (Hydra) gagne à passer par le glitch *en pratique* : une session où l'on corrompt un BMP dans Audacity, code un pixel sort en Python, simule un datamosh dans Hydra. Trois techniques, trois logiques, et une compréhension nouvelle de ce qu'est une image numérique.

1.2.5 Pour aller plus loin

- Rosa Menkman, *The Glitch Moment(um)*, INC, 2011 (téléchargeable libre).
- Iman Moradi, *Glitch Aesthetics*, mémoire de Master, 2004.
- Jon Cates, *Dirty New Media*, à partir de 2007 (manifestes en ligne).
- Daniel Temkin, blog *esoteric.codes*.
- Le festival *GLI.TC/H* (2010–2014, Chicago/Birmingham/Amsterdam) : archive en ligne.
- Kim Asendorf, *ASDFPixelSort* sur GitHub.
- Antonio Roberts, artiste glitch et chercheur, conférences *Glitch Aesthetics for Beginners*.

2 Le feedback en profondeur

Le feedback — la rétroaction — est l'un des grands principes esthétiques et techniques du temps réel. Une sortie qui revient en entrée, un système qui s'écoute lui-même, et qui dérive ou s'auto-stabilise. C'est aussi vieux que le micro qui siffle devant une enceinte, et aussi neuf qu'un `src(o0)` dans Hydra.

2.1 Origine cybernétique

Le terme *feedback* entre dans le langage scientifique avec Norbert Wiener, *Cybernetics : Or Control and Communication in the Animal and the Machine* (1948). Idée fondamentale : un système qui se régule en injectant son propre état dans son entrée. Deux régimes :

- **Feedback négatif** : la rétroaction *corrige* l'écart à une consigne (thermostat, régulateur de Watt). Stabilise.
- **Feedback positif** : la rétroaction *amplifie* l'écart. Diverge, sature, ou produit des oscillations.

L'art du feedback en arts numériques navigue presque toujours dans la zone *positive* ou à la frontière entre les deux. C'est là que la matière est intéressante : juste assez de divergence pour que le système ait l'air vivant, juste assez de stabilité pour qu'il ne sature pas instantanément.

2.2 Feedback audio

2.2.1 Feedback acoustique : le Larsen

Le cas canonique : un microphone capte une enceinte qui amplifie ce qu'il capte. À chaque tour de boucle, certaines fréquences (celles que la salle, le micro et l'enceinte privilégient) sont renforcées. Le système entre en oscillation à ces fréquences. C'est un système physique : la chaîne audio + l'acoustique de la pièce.

Jimi Hendrix. Au festival de Monterey en juin 1967, le solo de *Wild Thing* est joué avec la guitare collée à l'ampli, le Larsen comme matière musicale. Avant lui, Pete Townshend, Lou Reed, Sonic Youth plus tard, en feront un vocabulaire.

Glenn Branca, Rhys Chatham. Années 1980, New York : guitares amplifiées massivement, harmoniques générées par retour acoustique, devenues la base de symphonies pour ensembles de guitares électriques.

2.2.2 Alvin Lucier, *I Am Sitting in a Room* (1969)

La pièce-clé du feedback compositionnel. Lucier enregistre lui-même prononçant un texte, puis rejoue cet enregistrement dans la même pièce et le ré-enregistre. Il recommence. À chaque génération, les résonances propres de la pièce sont accentuées, la voix se désintègre, et il ne reste à la fin qu'une nappe harmonique — la *signature acoustique* de la pièce.

C'est un manifeste : *le médium est la composition*. Le code (Hydra, Strudel) peut faire la même chose — on y reviendra.

2.2.3 Stockhausen, *Mikrophonie I* (1964)

Deux musiciens frottent un grand tam-tam avec divers objets. Deux autres déplacent des micros directionnels près du tam-tam. Deux autres encore filtrent et spatialisent en direct. C'est un dispositif de feedback contrôlé : la sortie des micros est traitée et renvoyée en salle, où elle excite à nouveau le tam-tam.

2.2.4 No-input mixer (Toshimaru Nakamura)

Une table de mixage est branchée sur elle-même : sortie auxiliaire vers entrée d'un canal, avec bouclages internes. Aucun signal source. Le mixeur entre en oscillation et devient un instrument à part entière, dont on « joue » en bougeant les potentiomètres. Nakamura développe cette pratique dès la fin des années 1990.

2.2.5 Feedback synthétique : les bouclages internes

Dans les synthétiseurs et les langages de synthèse, les feedbacks numériques produisent une grande partie des sons les plus reconnaissables :

— **Filtre en peigne (*comb filter*)** : un délai très court rebouclé, qui crée des résonances harmoniques. Cœur des effets de « flanger », « chorus », et de la synthèse de Karplus-Strong.

— **Synthèse FM (Chowning, 1973)** : un oscillateur module la fréquence d'un autre. Si on reboucle un opérateur sur lui-même (auto-modulation), on obtient des spectres saturés caractéristiques (DX7, opérateur « feedback »).
— **Synthèse par modèles physiques** : on modélise l'instrument comme un système à feedback (corde + chevalet, anche + tube).
— **Karplus-Strong (1983)** : une corde simulée comme un délai bouclé sur lui-même avec un filtre passe-bas. Trois lignes de code donnent une corde pincée.

En SuperCollider — Karplus-Strong minimal.

```
// Une corde pincée par feedback dans un délai filtré
SynthDef(\karplus, { |freq=220, amp=0.3|
    var sig, in;
    in = WhiteNoise.ar() * EnvGen.kr(Env.perc(0.001, 0.05));
    sig = LocalIn.ar(1) * 0.99;       // lit le buffer feedback
    sig = LPF.ar(sig + in, freq * 4);
    sig = DelayC.ar(sig, 0.05, 1.0/freq);
    LocalOut.ar(sig);                  // écrit dans le buffer feedback
    Out.ar(0, sig * amp ! 2);
}).add;
```

Trois lignes de feedback (`LocalIn`, `DelayC`, `LocalOut`) suffisent à transformer un bruit blanc bref en corde pincée. Le coefficient `0.99` contrôle la décroissance ; `0.999` prolonge à l'infini.

Comb filter en Pure Data. Dans Pd, un objet `[delwrite~]` et un `[delread~]` avec un facteur de retour inférieur à 1 implémentent le même principe.

2.3 Feedback vidéo

2.3.1 Le feedback analogique : caméra-écran

Filmez un écran de TV qui affiche ce que la caméra filme. Au moindre angle, au moindre zoom, un motif émerge : couloirs infinis, fractales, vortex. C'est un feedback positif spatial. Steina et Woody Vasulka, Joan Jonas (*Vertical Roll*, 1972) ont systématiquement exploité cette boucle.

C'est aussi l'origine des *video synthesizers* (Sandin, Rutt-Etra) : on patche des entrées vidéo dans des modulateurs et on reboucle.

2.3.2 Feedback numérique : Hydra

Hydra a fait du feedback un primitif central avec `src(o0)` (lire le buffer de sortie comme une source). Trois exemples progressifs :

Feedback simple — le couloir infini.

```
osc(10, 0.1, 0.8)
  .out()

src(o0)
  .scale(0.98)        // chaque image est légèrement réduite
```

```
  .layer(osc(10, 0.1, 0.8).mask(shape(4, 0.3)))
  .out()
```

À chaque frame, on reprend l'image précédente, on la rétrécit légèrement, et on superpose une nouvelle source. Le résultat : un puits visuel.

Feedback modulé — le « Lucier visuel ».

```
// Comme Lucier mais en image : on copie la sortie sur elle-même
// avec des transformations qui s'accumulent.
osc(20, 0.05, 1.4)
  .modulate(src(o0), 0.005)
  .blend(src(o0), 0.92)
  .out()
```

Le `blend(src(o0), 0.92)` mélange à 92% l'image précédente avec la nouvelle : les structures persistent, se traînent, lissent les couleurs en grandes nappes.

Feedback chaotique.

```
src(o0)
  .rotate(() => 0.01 + a.fft[0]*0.05)
  .scale(1.01)
  .modulateHue(src(o0), 0.4)
  .layer(osc(40, 0.1).luma(0.3).color(1, 0, 0))
  .out()
```

Chaque frame est tournée, agrandie, et sa teinte modulée par elle-même. Sans la couche `layer`, le système saturerait en quelques secondes ; cette injection régulière de « matière fraîche » le maintient au seuil.

2.3.3 Concept clé : le coefficient de fuite

Tout feedback artistique repose sur un dosage entre *persistance* (combien on garde du passé) et *nouveauté* (combien on injecte). En Hydra, c'est le facteur du `blend` ou du `scale`. En audio, c'est le coefficient de feedback du délai. C'est le seul réglage important, mais il est sensible : 0.95 et 0.99 ne donnent pas du tout la même musique.

2.4 Feedback en code : auto-modification

Une frontière plus radicale : un programme qui modifie son propre code. Peu de systèmes permettent ça proprement, mais quelques expériences existent :

- **Extempore** (Andrew Sorensen) permet de redéfinir des fonctions pendant qu'elles sont en cours d'exécution, et même de programmer un agenda où une fonction s'auto-réécrit après quelques secondes.
- **Gibber** (Charlie Roberts) intègre des notions de « réflexivité » où l'éditeur lui-même peut être modifié par le code joué.

C'est la limite asymptotique du live coding : le code n'est plus seulement exécuté, il s'auto-réécrit. La performance devient une boucle d'écriture-réécriture qui vit toute seule.

2.5 Feedback comme matière mathématique

2.5.1 La carte logistique (May, 1976)

$$x_{n+1} = r \cdot x_n \cdot (1 - x_n)$$

Un système discret à feedback, dont le comportement varie radicalement selon le paramètre `r` :

— `r < 3` : convergence vers un point fixe.
— `3 < r < 3.45` : oscillation entre deux valeurs.
— `r > 3.57` environ : régime chaotique, sensibilité aux conditions initiales.

C'est un excellent générateur de signal de contrôle pour un live codeur :

```
// SuperCollider : carte logistique comme LFO
SynthDef(\logistic_lfo, { |out=0, r=3.8, freq=4|
    var x;
    x = LocalIn.kr(1, 0.5);
    x = r * x * (1 - x);
    LocalOut.kr(x);
    Out.kr(out, x);
}).add;
```

Branche cette sortie sur n'importe quel paramètre (cutoff, rate, amplitude) : selon `r`, tu as un LFO sage ou un signal chaotique.

2.5.2 L'attracteur de Lorenz (1963)

Trois équations différentielles couplées :

$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z$$

Avec $\sigma = 10, \rho = 28, \beta = 8/3$, le système trace l'*attracteur étrange* en forme de papillon. Les trois variables x, y, z font trois signaux de contrôle corrélés mais imprévisibles — parfait pour piloter trois paramètres d'un visuel ou trois voix d'un synthé.

2.6 Feedback : pratique pédagogique

On peut imaginer un atelier dédié au feedback en **trois temps** :

1. **Acoustique.** Un micro, une enceinte, on écoute le Larsen, on apprend à le contrôler en bougeant le micro. Tout part de là.
2. **Numérique audio.** En SuperCollider ou Sonic Pi, on construit un Karplus-Strong, on bouge le coefficient de feedback de 0.5 à 0.999, on entend la corde apparaître, puis exploser.
3. **Numérique visuel.** En Hydra, on part de `osc().out()`, on ajoute `src(o0)`, on dose le `blend`, on observe.

À la fin, on comprend que tous ces phénomènes sont *le même geste* sur des médiums différents.

2.7 Pour aller plus loin

— Norbert Wiener, *Cybernetics*, 1948.
— Alvin Lucier, *Reflections / Réflexions*, MusikTexte, 1995.
— David Toop, *Sinister Resonance*, Continuum, 2010.
— James Mooney, *Sound, Music and Resonance : A Cultural History of Listening to Air*, 2018.
— Sur la carte logistique : Robert M. May, « Simple Mathematical Models with Very Complicated Dynamics », *Nature*, 1976.
— Sur Karplus-Strong : Karplus & Strong, « Digital Synthesis of Plucked-String and Drum Timbres », *Computer Music Journal*, 1983.
— À voir : la captation de *I Am Sitting in a Room* sur le site officiel d'Alvin Lucier ; *Bird's Eye* de Steina Vasulka.

3 Pure Data : programmation visuelle pour le temps réel

3.1 Présentation

Pure Data (souvent abrégé *Pd*) est un environnement de programmation visuelle pour le son, l'image et l'interactivité, créé par **Miller Puckette** en 1996. Il appartient à une autre famille que les outils textuels du cursus : on y « patche » en branchant des boîtes les unes aux autres, comme on connecterait des modules eurorack.

C'est l'outil de référence des arts numériques depuis vingt-cinq ans : installations interactives, scénographies, instruments embarqués, prototypage temps réel. Sa philosophie est complémentaire à celle du live coding textuel — et c'est précisément pour ça qu'il a sa place dans un cursus complet.

3.2 Filiation : Patcher, Max, Pd

L'histoire est celle d'une bifurcation :

1. **1985–1990, IRCAM, Paris.** Miller Puckette développe *Patcher* sur Macintosh, première interface visuelle pour piloter le système 4X de l'IRCAM. C'est le grand-père commun.
2. **1990, IRCAM puis Cycling '74.** Patcher devient *Max* (en hommage à Max Mathews). Une partie devient commerciale — aujourd'hui *Max/MSP/Jitter* édité par Cycling '74 (David Zicarelli).
3. **1996, UCSD.** Puckette repart d'une page blanche, plus modulaire et libre : *Pure Data*. Open source dès l'origine.

Pd et Max sont conceptuellement frères : presque les mêmes objets fondamentaux, presque la même logique de patch. Mais Pd est libre, Max est payant et plus poli graphiquement.

3.3 Versions

— **Pd Vanilla** : la distribution officielle de Puckette. Minimaliste, stable, peu d'objets externes intégrés.
— **Pd Extended** : distribution communautaire enrichie de bibliothèques. *Abandonnée depuis 2013*, à éviter sur les nouveaux systèmes.

— **Purr Data** : fork actif de Pd Extended (Jonathan Wilkes). Compatibilité élargie, interface modernisée.
— **PlugData** (Timothy Schoen, 2022–) : recommandé aujourd'hui pour les débutants. Interface moderne, fonctionne en plugin VST/AU dans une DAW, tout en restant 100% compatible Pd.

Pour le cursus : **PlugData**.

3.4 Concepts fondamentaux

3.4.1 Patch

Un programme Pd est un *patch* : un canevas où l'on dispose des boîtes (objets, messages, nombres, atomes) reliées par des cordons.

3.4.2 Types de boîtes

— **Objets** : `[osc~ 440]`, `[+ 1]`, `[delay 500]`. Une fonction. Le ~ en suffixe indique un objet *audio rate* (signal continu) ; sans ~, c'est du *message rate* (évènements ponctuels).
— **Messages** : `[hello]`, `[set 100]`. Données envoyées à la demande.
— **Nombres** : `[number]`, modifiables à la souris.
— **Symboles, sliders, radio buttons, bangs**, etc.

3.4.3 Inlets et outlets

Chaque boîte a des entrées (*inlets*, en haut) et des sorties (*outlets*, en bas). Règle fondamentale :

— L'**inlet de gauche est « chaud »** : tout message reçu déclenche le calcul.
— Les **inlets suivants sont « froids »** : ils mettent à jour des valeurs internes mais ne déclenchent rien.

Conséquence pratique : pour qu'un `[+]` additionne A et B, il faut envoyer B en froid d'abord, puis A en chaud.

3.4.4 Audio rate vs message rate

— **Message rate** : asynchrone, événementiel. Un *bang*, un nombre, une note MIDI.
— **Audio rate** (~) : flux continu à la fréquence d'échantillonnage (44100, 48000 Hz). Cordons jaunes sur les versions modernes.

Cette dualité *message / signal* structure tout Pd. Comprendre où se fait la frontière (objets de conversion comme `[snapshot~]`, `[sig~]`) est l'essentiel.

3.5 Premier patch : un oscillateur

Voici à quoi ressemble un patch minimal en pseudo-ASCII (équivalent textuel d'un patch graphique) :

```
[number 440]              <-- fréquence ajustable
     |
[osc~ 440]                <-- oscillateur sinusoïdal audio
     |
```

```
   [*~ 0.3]                  <-- multiplication par 0.3 (volume)
        |
   [dac~]                    <-- sortie audio (digital-to-analog converter)
```

Quatre boîtes, trois cordons, on entend un sinus à 440 Hz. Modifier le `number` change la fréquence en temps réel.

3.6 Patch légèrement enrichi : un FM minimal

```
[number 220]   [number 660]
     |              |
   [osc~]         [osc~]
                    |
                  [*~ 100]      <-- index de modulation
                    |
                  [+~]<-------|  <-- somme freq porteuse + modulateur
                    |
                  [osc~]          <-- oscillateur porteur
                    |
                  [*~ 0.3]
                    |
                  [dac~]
```

(Ce diagramme est approximatif — il faut un `[+ 220]` pour additionner la fréquence porteuse fixe au signal modulateur audio. À voir en cours sur l'éditeur réel.)

3.7 Feedback en Pd

Le feedback est aussi un primitif facile : on connecte une sortie à une entrée précédente. Pour des feedbacks audio sans larsen numérique, on utilise les paires `[send~ bus]` / `[receive~ bus]` qui découplent visuellement les cordons.

Karplus-Strong en Pd : `[noise~]` pulse bref → `[delwrite~ corde 50]` (50 ms de délai) ← `[delread~ corde]` multiplié par 0.99 et filtré, rebouclé en `[delwrite~]`. Six boîtes, une corde.

3.8 GEM et le visuel

GEM (*Graphics Environment for Multimedia*) est l'extension graphique historique de Pd, ajoutée par Mark Danks dès 1995. Permet de manipuler du 2D, 3D, vidéo, OpenGL en patch. Aujourd'hui en concurrence avec **Ofelia** (intégration d'openFrameworks) et avec des solutions externes via OSC vers Hydra/TouchDesigner.

3.9 Cas d'usage typiques

— **Installations interactives.** Capteurs (Arduino, Kinect, OSC) → Pd → son et image. La majorité des installations arts numériques tournent ou ont tourné sur Pd ou Max.
— **Performance live électronique non-textuelle.** Beaucoup d'artistes préfèrent un patch préparé qu'ils modulent en live avec contrôleurs.
— **Embarqué.** `libpd` (Peter Brinkmann, 2010) permet d'embarquer un moteur Pd dans une appli mobile, un Raspberry Pi, un Bela. Beaucoup de projets de hardware musical l'utilisent.

— **Recherche musicale.** Pd reste un standard à l'IRCAM, à STEIM, dans les laboratoires académiques.

3.10 Pure Data et live coding : le faux opposé

Pd est souvent présenté comme l'opposé du live coding textuel :

Pd / Max	**Live coding textuel**
Programmation visuelle	Programmation textuelle
Patch préparé, modulé en live	Code écrit/modifié pendant la performance
Le geste est dans les contrôleurs	Le geste est dans la frappe
Le « programme » est figé visuellement	Le programme est en mouvement permanent

Mais cette opposition se nuance en pratique :

— Pd permet de modifier le patch *en cours d'exécution* : ajouter des boîtes, en supprimer, redessiner des cordons. C'est rare en performance, mais ça existe.
— Inversement, beaucoup de live codeurs textuels travaillent à partir de « templates » préparés.
— Des outils hybrides comme **Bespoke Synth** (Ryan Challinor, 2020) ou les *node-based* comme **vvvv**, **TouchDesigner** brouillent encore plus la frontière.

Penser Pd comme un *cousin du live coding* plus qu'un opposé. Le geste mental — modulariser le son comme une chaîne de transformations — est exactement le même qu'en SuperCollider ou en Strudel.

3.11 Place dans le cursus

Pd mérite **une séance dédiée**, idéalement après avoir vu la pensée « patch » en SuperCollider et avant Hydra. Logique : montrer que les langages textuels et visuels manipulent les mêmes idées.

Une séance type (3h) :

1. Tour rapide de l'interface PlugData (30 min).
2. Construction d'un oscillateur, d'un filtre, d'un délai (1h).
3. Construction d'un Karplus-Strong en patch (45 min).
4. Pont avec SuperCollider via OSC (45 min).

3.12 Pour aller plus loin

— Site officiel : `https://puredata.info/`.
— PlugData : `https://plugdata.org/`.
— Documentation officielle : *Pd Manual* de Miller Puckette (téléchargeable, accessible aux débutants).
— Andy Farnell, *Designing Sound*, MIT Press, 2010 — la référence pour la synthèse procédurale en Pd. Construit un cours complet de synthèse à travers des patchs.
— Johannes Kreidler, *Loadbang : Programming Electronic Music in Pd*, 2009 (téléchargeable libre). Court et excellent pour démarrer.
— Floss Manual *Pure Data* (en français, en partie) : `https://flossmanuals.net/puredata-fr/`.
— Communauté francophone : forum *Pure Data France*, communauté Discord.

4 Open source, art libre et Creative Commons

Cette section n'est pas un appendice juridique. C'est l'ossature *politique et éthique* du live coding, qu'il faut transmettre en même temps que la technique. Faire du live coding sans comprendre la culture du logiciel libre, c'est manquer la moitié du sujet.

4.1 Free software : l'origine politique

4.1.1 Richard Stallman et le projet GNU (1983)

Stallman, chercheur au MIT AI Lab, démissionne en 1983 et lance le projet **GNU** (*GNU's Not Unix*) avec pour but de construire un système d'exploitation entièrement libre. En 1985, il fonde la **Free Software Foundation**.

Sa thèse : un logiciel propriétaire est *politiquement* problématique, parce qu'il enchaîne l'utilisateur. Il définit le **logiciel libre** par quatre libertés (numérotées 0 à 3, comme un programmeur) :

- **Liberté 0** : exécuter le programme, pour tout usage.
- **Liberté 1** : étudier le programme, le modifier — ce qui suppose l'accès au code source.
- **Liberté 2** : redistribuer des copies.
- **Liberté 3** : distribuer des copies modifiées.

Le mot *free* est ici « libre » (comme *freedom of speech*), pas « gratuit » (*free beer*). La distinction « libre vs gratuit » est essentielle.

4.1.2 Copyleft et licence GPL

La licence **GPL** (*GNU General Public License*, 1989, v2 en 1991, v3 en 2007) introduit le **copyleft** : tout dérivé d'une œuvre GPL doit lui-même être distribué sous GPL. Symbole : un « C » retourné. C'est une logique *virale* et délibérée : la liberté se propage.

4.2 Open source : la reformulation pragmatique (1998)

En 1998, Eric Raymond, Bruce Perens et d'autres lancent l'**Open Source Initiative** (OSI). Mêmes pratiques techniques, mais un discours plus pragmatique : *l'open source, c'est efficace*. Idée centrale formulée par Raymond dans *The Cathedral and the Bazaar* (1999) : « Given enough eyeballs, all bugs are shallow. »

Différence philosophique avec Stallman :

- **Free software** (FSF) : argument éthique, la liberté d'abord.
- **Open source** (OSI) : argument pragmatique, la qualité d'abord.

En pratique, les listes de licences agréées par les deux organisations se recouvrent à 99%. La différence est rhétorique mais elle compte pour comprendre les débats internes au mouvement.

4.3 Familles de licences logicielles

4.3.1 Copyleft fort

- **GPL v2 / v3** : le dérivé doit être GPL. SuperCollider est en GPL.
- **AGPL** : étend la GPL au cas du logiciel servi via un réseau (anti-loophole « SaaS »).

4.3.2 Copyleft faible

— **LGPL** : les bibliothèques peuvent être linkées par du code propriétaire, mais leurs modifications restent LGPL.
— **MPL (Mozilla)** : copyleft fichier-par-fichier.

4.3.3 Permissif

— **MIT**, **BSD** : on peut tout faire, y compris fermer un dérivé. Pure Data, Hydra sont sous des licences permissives proches.
— **Apache 2.0** : permissif avec clauses sur les brevets.

Le choix entre copyleft et permissif n'est pas neutre. Copyleft : on protège la liberté du futur. Permissif : on maximise l'adoption, quitte à ce qu'un acteur commercial enferme un dérivé.

4.4 Les outils du cursus, leurs licences

Outil	Licence	Famille
SuperCollider	GPL v3	Copyleft fort
Sonic Pi	MIT	Permissif
TidalCycles	GPL v3	Copyleft fort
Strudel	AGPL v3	Copyleft fort + réseau
Hydra	AGPL v3	Copyleft fort + réseau
Pure Data	BSD-3	Permissif
Resolume	Propriétaire	—

Six outils sur sept de l'écosystème live coding sont libres. Le seul propriétaire (Resolume) est aussi le seul qui n'est pas issu de la culture live coding mais de l'industrie VJ.

4.5 Creative Commons : libérer les œuvres, pas les programmes

Les licences libres logicielles ne couvrent pas bien les *œuvres artistiques* (musique, image, vidéo, texte). En 2001, Lawrence Lessig (juriste, professeur à Stanford) fonde **Creative Commons** pour combler ce manque.

Une licence CC est une combinaison de quatre clauses :

— **BY** (Attribution) : créditer l'auteur. Toujours présente.
— **SA** (Share-Alike) : les dérivés doivent être sous la même licence (équivalent du copyleft).
— **NC** (Non-Commercial) : interdit l'usage commercial.
— **ND** (No Derivatives) : interdit les dérivés.

D'où **six combinaisons**, classées de la plus libre à la plus restrictive :

1. **CC BY** — liberté quasi totale, juste créditer.
2. **CC BY-SA** — liberté + copyleft. C'est la licence de Wikipédia.
3. **CC BY-NC** — liberté pour usage non commercial.
4. **CC BY-NC-SA** — non commercial + copyleft.
5. **CC BY-ND** — distribution sans modification.
6. **CC BY-NC-ND** — la plus restrictive, équivaut presque à « droits réservés ».

Et hors combinaisons : **CC0** — renonciation au copyright, équivalent du domaine public.

4.5.1 Critique des clauses NC et ND

Beaucoup de défenseurs du libre sont méfiants envers **NC** et **ND** :

— **NC** est juridiquement flou (qu'est-ce qu'un « usage commercial » ? Une bibliothèque ? Un blog avec une publicité ?). En pratique, NC freine la circulation autant qu'un copyright classique.
— **ND** interdit le remix — ce qui va contre l'esprit même des arts numériques où l'œuvre est souvent matière à transformation.

Pour un live codeur qui partage son code et ses captations, **CC BY** ou **CC BY-SA** sont les choix naturels. Un set Algorave publié sur YouTube en CC BY-SA est cohérent avec le manifeste TOPLAP.

4.6 Free Art License (LAL)

Moins connue mais importante en France : la **Licence Art Libre** (LAL), publiée en 2000 par le collectif *Copyleft Attitude* à Paris. Conçue *spécifiquement pour l'art*, elle est un copyleft pur appliqué aux œuvres : tout dérivé doit lui aussi être en LAL ou compatible.

Compatible avec CC BY-SA depuis 2014. À mentionner parce qu'elle vient d'une réflexion européenne et artistique, pas seulement logicielle.

4.7 Open art, art libre, art en partage

Au-delà des licences, une **culture du partage** traverse les arts numériques. Quelques formes concrètes :

— **Code source publié.** GitHub, Codeberg, Bitbucket regorgent de patchs Pd, sketches Hydra, projets SuperCollider. Tu peux récupérer le set d'Alex McLean joué à un Algorave et l'étudier.
— **Captations en ligne.** La plupart des Algoraves, des Chaos Labs, des conférences live coding sont diffusées sur YouTube/PeerTube avec un timestamp.
— **Workshops gratuits.** TOPLAP organise régulièrement des ateliers d'initiation gratuits ; même logique pour Sonic Pi (Live Coding Education) et Strudel (les workshops embarqués dans `strudel.cc`).
— **Forums et chats horizontaux.** *TidalCycles Discord*, *Sonic Pi Slack*, *TOPLAP Mattermost*. Pas de gatekeeping, peu de hiérarchie.

4.8 Limites et tensions

Une transmission honnête doit aussi nommer les problèmes :

— **Travail invisible**. Maintenir un outil open source comme SuperCollider, c'est une charge énorme portée par une poignée de personnes, souvent gratuitement. Le sujet du *burnout open source* est documenté.
— **Modèles économiques**. Comment vit-on de l'art libre ? Réponses partielles : performance live (où la rareté de l'évènement est protégée), résidences, financements publics, donations (*ko-fi*, *Liberapay*, *GitHub Sponsors*), enseignement.
— **NC et l'ambiguïté**. La majorité des artistes choisissent NC « par défaut » sans réaliser qu'ils restreignent fortement la diffusion. Discussion à avoir avec ton élève.
— **Limites du copyleft musical**. La SACEM française n'accepte pas les œuvres sous licence libre dans son catalogue (sauf via la Société Générale du Domaine Public ou *Dogmazic*). Question pratique pour qui veut diffuser commercialement.

4.9 Qu'enseigner concrètement

Quelques recommandations à transmettre :

1. **Lis les licences des outils que tu utilises.** Cinq minutes de lecture pour comprendre dans quoi tu t'inscris.
2. **Choisis une licence pour ton propre travail dès le début.** Par défaut : **CC BY-SA 4.0** pour les œuvres, **MIT** ou **GPL v3** pour le code, selon que tu veux maximiser l'adoption ou protéger la liberté du dérivé.
3. **Crédite systématiquement.** Mentionner Olivia Jack quand tu publies un set Hydra, c'est une question de respect minimal.
4. **Publie le code de tes performances.** Pas « quand ce sera propre », dès maintenant. La culture du *commit* régulier est plus saine que celle du *drop* parfait.
5. **Utilise des plateformes neutres.** GitHub appartient à Microsoft. Codeberg (allemand, non-profit) ou un Gitea auto-hébergé sont des options plus alignées.

4.10 Pour aller plus loin

— Richard Stallman, *Free Software, Free Society*, FSF, 2010 (recueil d'essais, librement disponible).
— Lawrence Lessig, *Free Culture*, 2004 (téléchargeable gratuitement). La défense la plus claire de Creative Commons.
— Lawrence Lessig, *Code is Law : On Liberty in Cyberspace*, *Harvard Magazine*, 2000.
— Eric S. Raymond, *The Cathedral and the Bazaar*, 1999 — du côté open source.
— Christopher Kelty, *Two Bits : The Cultural Significance of Free Software*, Duke, 2008. Analyse anthropologique du mouvement.
— Florent Latrive, *Du bon usage de la piraterie*, La Découverte, 2004 (contexte francophone).
— Site officiel Creative Commons : `https://creativecommons.org/`.
— Outil de choix de licence : `https://choosealicense.com/`.
— Licence Art Libre : `https://artlibre.org/`.